PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026
Email: umhackathon@um.edu.my

Table of Contents

1. Project Overview 3

2. Background & Business Objective 3

3. Product Purpose 4

4. System Functionalities 5

5. User Stories & Use Cases 6

6. Features Included (Scope Definition) 8

7. Features Not Included (Scope Control) 8

8. Assumptions & Constraints 8

9. Risks & Questions Throughout Development 9

© UMHackathon 2026
Email: umhackathon@um.edu.my

Project Name: SCADA-i

Version: 3.0

1. Project Overview

- **Problem Statement:** Facility managers and building administrators struggle to manually synthesize vast amounts of raw IoT data into actionable insights, compliance reports and operational policies. Furthermore, coordinating workspace bookings with real-time room availability currently requires constant human oversight.
- **Target Domain:** Smart Building Management System (SCADA-i)
- **Proposed Solution Summary:** SCADA-i extends traditional facility monitoring systems by introducing a supervisory multi-agent AI architecture. Specialized agents are responsible for domain-specific functions such as HVAC optimization, room booking management and ESG compliance monitoring. A higher-level Z.AI GLM orchestrator coordinates these agents by interpreting user inputs, performing reasoning and planning, and delegating tasks gracefully. This enables automated workflows such as generating ESG reports, issuing maintenance tickets and managing workspace bookings through an integrated web-based interface.

2. Background & Business Objective

- **Background of the Problem:** For many years, traditional Building Management Systems (BMS) and SCADA dashboards primarily function as passive monitoring tools, providing raw sensor data such as temperature, occupancy and equipment status. While these systems offer visibility, they lack the capability to interpret data, make decisions or execute actions autonomously. As a result, facility managers are required to manually analyze large volumes of IoT data, identify inefficiencies and anomalies, generate sustainability (ESG) reports, coordinate maintenance activities and finally, manage room bookings and occupancy. This manual labor ultimately leads to energy waste, undetected

© UMHackathon 2026
Email: umhackathon@um.edu.my

issues such as sensor failures or ESG policy violations, high administrative overhead for monitoring and reporting, and finally poor coordination across building functions (HVAC, booking, ESG).

- **Importance of Solving This Issue:** By automating the interpretation of real-time IoT data and converting it into executable actions, SCADA-i enables real-time optimization of energy usage, early detection of operational issues before they become costly, automated ESG compliance monitoring and reporting with minimal manual effort, and most importantly, seamless coordination of building operations.
- **Strategic Fit / Impact:** It aligns with smart building initiatives, global sustainability goals and ESG compliance. By introducing a multi-agent AI system with a central orchestrator, SCADA-i forms a closed loop intelligent system where facility-owners can effortlessly hit their carbon-reduction targets, streamline their compliance reporting and offer a seamless environment for workspace users.

3. Product Purpose

3.1. Main Goal of the System: To provide an autonomous, AI-driven facility management copilot that transforms passive IoT infrastructure into an agentic system. It optimizes building energy usage while maximizing occupant user experience and comfort through dynamic reasoning and hardware orchestration.

3.2. Intended Users (Target Audience):

- Students and Campus Staff
- Campus Facility Managers
- Maintenance Technicians
- Clients and Office Employees

© UMHackathon 2026

4. System Functionalities

4.1. Description: SCADA-i operates as a multi-agent AI workflow system that transforms unstructured user inputs and real-time IoT data into actionable decisions and automated execution. User requests are processed by a central Z.AI GLM orchestrator, which interprets intent, retrieves relevant system context and delegates tasks to specialized agents (HVAC, Booking, ESG). These agents coordinate with backend services and IoT infrastructure to execute real-world actions. Unlike traditional systems that rely on static schedules, SCADA-i enables a closed-loop workflow where data is continuously monitored, decisions are made in real time and actions are executed automatically.

4.2. Key Functionalities:

- **Intelligent Energy Optimization:** The Orchestrator delegates tasks to the HVAC Agent, which evaluates real-time room conditions, occupancy and ESG constraints. Then the HVAC agent applies policy-aware temperature control, uses a LightGBM model optimizer for continuous fine-tuning and finally executes HVAC commands via MQTT. Our system also supports manual override with temporary optimizer pause through natural language or admin web-based dashboard.
- **Smart Booking & Space Management:** Users can interact with the system using natural language to request room bookings. Then, the booking agent queries room availability from the database and also checks for requested equipment. Next, it validates any scheduling conflicts and updates booking records in the database. Furthermore, our system can also detect upcoming bookings automatically and execute HVAC pre-cooling in advance.
- **Automated ESG Monitoring & Reporting:** The ESG Agent continuously analyzes system data to ensure sustainability compliance. It also monitors energy usage and environmental metrics, detects policy violations and anomalies and lastly generates structured ESG reports on demand for compliance purposes.

© UMHackathon 2026

4.3. AI Model & Prompt Design

4.3.1. Model Selection: The deployed orchestrator uses OpenAI 'gpt-4o' as the central reasoning/router model because it reliably produces structured routing outputs for multi-agent delegation. Domain workers (Booking, HVAC, ESG) run on 'deepseek-chat' for tool-calling execution to keep blended inference cost lower.

4.3.2. **Prompting Strategy:** We utilized a Hierarchical Orchestrator-Worker prompting strategy where the Orchestrator interprets user intent and delegates explicit commands. Then, specialized workers execute domain tools and return structured reports. This decomposition prevents a single monolithic prompt from being overloaded by cross-domain building-management tasks.

4.3.3. **Context & Input Handling**: To reduce context rot and control token growth, the system applies Just-in-Time (JIT) context trimming using recency-based message selection (`trim_messages` with `max_tokens=4000`, `strategy="last"`) before supervisor/synthesizer calls. This 4,000-token bound is a per-invocation prompt budget, not a strict end-to-end session cap. System instructions are re-applied every invocation, while older chat turns are trimmed when needed.

4.3.4. **Fallback & Failure Behavior:** Reliability/safety is enforced through 3 stages. First, supervisor fallback behavior (route to synthesizer on routing exceptions). Next, domain policy controls (e.g., HVAC requests outside allowed temperature bounds are automatically clamped to policy-compliant values before actuation). Finally, operational fail-safe expectation (human escalation and safe-state procedures are defined at deployment level).

5. User Stories & Use Cases

- **User Stories:**
  - *As a student, I want to text the campus bot that my study group is feeling hot. I want it to adjust the AC without needing to find a physical remote or understand the complex BMS app.*

© UMHackathon 2026

- *As a facility manager, I want the system to automatically detect broken AC units and turn them off, so that energy isn't wasted and electrical hazards are prevented*
- *As an office employee, I want to text the bot to prepare a meeting room for an impromptu client pitch, so that I don't have to waste time searching for an empty room or waiting for the AC to cool ahead of time.*
- *As a sustainability officer, I want to generate ESG reports quickly, so that I can track energy usage and compliance without manual analysis.*

- **Use Cases (Main Interactions):**

Smart Booking & Client Prep: User submits a request like *"Book a room for 6 people"*. The Orchestrator (GLM) first interprets user intent, next identifies it as a booking request. It then assigns the booking agent to query the room availability database, suggests suitable room types and based on the user's additional request/response it creates a booking record stored in the database. Then, the HVAC agent is triggered to calculate pre-cooling requirements (if requested at an AC setpoint) and updates the pre-cooling schedule. Finally, in the execution layer, the scheduler triggers HVAC adjustment before booking time and the orchestrator returns confirmation to user

Energy Optimization (System Triggered): Sensor detects occupancy drop which the Orchestrator receives and determines an optimization opportunity. It calls the HVAC agent to evaluate current room conditions and adjusts temperature setpoint. The optimizer (LightGBM) then refines HVAC parameters and finally, the execution layer sends updated HVAC commands via MQTT. Last of all, the system updates room state in the database.

ESG Reporting: Admin requests *"Generate weekly ESG report"*. The Orchestrator interprets requests, reasons and decides to delegate to the ESG agent. The ESG agent retrieves current week's sensor data and calculates energy savings and emissions. Finally the orchestrator generates a structured report and the report is returned in PDF format.

© UMHackathon 2026
Email: umhackathon@um.edu.my

6. Features Included (Scope Definition)

- Natural-language web interaction interface for facility users/operators to submit requests and receive guided outcomes.
- Multi-agent AI system (Orchestrator, HVAC Agent, Booking Agent, ESG Agent) powered by Z.AI ILMU-GLM-5.1 for decision-making and task delegation.
- Closed-loop operational flow (e.g., request intake → policy/context checks → decision → command execution → status feedback)
- Safety guardrails through policy-bounded control logic to prevent unsafe HVAC setpoints and reject/adjust invalid requests.
- IoT integration via MQTT for HVAC command/control and optimizer communication.
- Hybrid HVAC optimization combining rule-based control, occupancy logic and optimizer-driven recommendations (including pre-cooling workflows).
- Room availability and booking lifecycle actions (check, create, update/reschedule, cancel) through tool-driven execution.
- Capacity/attribute-aware booking flow where room directory constraints are checked before finalizing availability decisions.
- Post-booking HVAC handoff where successful booking confirmation can trigger pre-cooling scheduling.
- Policy-aware HVAC compliance checks before temperature actions are executed.
- Retrieval-augmented policy lookup using embeddings + vector search to ground ESG decisions on indexed policy sources.

© UMHackathon 2026

- Role-aware governance behavior for restricted reporting access and explicit denial responses for unauthorized users.

- Compliance reporting support for ESG-related operational visibility within the current tool scope.

7. Features Not Included (Scope Control)

- Installation of new IoT sensors, hardware or edge nodes across the entire building/campus (testing is strictly limited to the MVP prototype environments).

- Multi-building or campus-wide deployment with centralized monitoring dashboard.

- Voice-to-text processing, speech recognition or audio-based interactions.

- Advanced ESG reporting dashboards with analytics and visualization.

8. Assumptions & Constraints

- **LLM Cost Constraint:** We estimate an average user session consumes roughly 6,000–9,000 total tokens (input + output across supervisor and worker agents). To keep inference cost manageable at scale, we made several architecture decisions:
    - Cap active prompt context with message trimming (max_tokens=4000).
    - Use stronger models only for routing while using lower-cost tool workers for execution.
    - Enforce recursion limits to prevent runaway tool loops.
    - Use retrieval (Top-k Policy Chunks) instead of full-document prompting for ESG checks.
- **Technical Constraints:** Current capabilities are domain-scoped (Booking + HVAC + ESG), depend on external services (LLM APIs, Supabase, Postgres, MQTT) and are coupled to MQTT protocol contracts for HVAC actuation. User interfaces are prototype-oriented rather than full enterprise production consoles.

© UMHackathon 2026
Email: umhackathon@um.edu.my

- **Performance Constraints:** We cap prompt context for speed/cost, which can reduce long-session memory fidelity. Recursion limits may truncate very complex multi-step tasks, and HVAC responsiveness depends on MQTT/network timeout behavior. ESG compliance latency and quality also depend on retrieval/index coverage.
- **User Input:** The system requires user/system prompts to initiate actions, requires clarifying input for incomplete booking requests (time/duration/capacity), enforces role-based restrictions for sensitive reporting and still assumes human oversight before high-impact operational actions in production contexts.

9. Risks & Questions Throughout Development

- **AI Decision Reliability**
    - How can we ensure that the LLM consistently produces accurate and grounded decisions?
    - What happens if the model generates incorrect or unsafe HVAC commands?
- **System Stability & Failure Handling**
    - How does the system behave if MQTT communication fails or is delayed?
    - What happens if the optimizer does not respond within the expected time?
- **Input Interpretation & Ambiguity**
    - How does the system handle ambiguous or incomplete user inputs?
    - What happens if the orchestrator misinterprets user intent and triggers the wrong agent?
- **Data Integrity & Sensor Reliability**
    - How does the system respond to missing or inconsistent sensor data?
    - Can faulty sensor readings lead to incorrect system decisions?
- **Dependency on External Services**
    - What is the impact if the Z.AI GLM API becomes unavailable or rate-limited?
    - How can the system maintain safe operation without the AI layer?

© UMHackathon 2026